Java Programming Homework | Characters, Strings, and StringBuilder, array

For each question, write, compile, and test a Java class. Save each class in a separate .java file using the filename shown.

Question 1: Password Strength Checker

Write a Java program that checks the strength of several passwords. The program first reads an integer n, representing the number of passwords. It then reads n passwords, one per line. Each password contains between 1 and 50 printable ASCII characters without spaces.

A password is classified as Strong if it satisfies all the following conditions:

- It contains at least eight characters.
- It contains at least one uppercase letter.
- It contains at least one lowercase letter.
- It contains at least one digit.
- It contains at least one of these special characters: !@#$%&*.

A password that fails any condition is classified as Weak. Other characters are allowed but do not satisfy the special-character requirement.

Use String methods and character-checking methods from the Character class to examine each password. Display its classification on a separate line. Finally, display the total number of strong passwords.

Assume that $1 \leq n \leq 20$. Read from standard input without displaying input prompts.

Question 2: Sentence Normalizer

Write a Java program that normalizes a single line of text. The input contains between 1 and 200 characters and consists only of English letters and spaces. At least one letter is guaranteed.

Normalize the text using these rules:

- Remove all leading and trailing spaces.
- Replace each sequence of consecutive spaces with one space.
- Convert the first letter of each word to uppercase.
- Convert all remaining letters in each word to lowercase.

Use a StringBuilder to construct the normalized sentence. Examine the input character by character, and use methods from the Character class when changing letter case. Do not use regular expressions or the split() method.

Display the normalized sentence on the first line. On the second line, display the number of words using the label Words: . A word is a consecutive sequence of letters separated from other words by spaces.

Read the entire input line from standard input without displaying an input prompt.

Question 3: Letter Frequency Report

Write a Java program that counts how often each English letter appears in a line of text. The input contains between 1 and 200 printable ASCII characters and may include uppercase letters, lowercase letters, digits, spaces, and punctuation.

Treat uppercase and lowercase versions of the same letter as identical. Ignore all characters that are not English letters.

Use an integer array of length 26 to store the frequencies, with index 0 representing a and index 25 representing z. Examine the input one character at a time.

Display each letter that appears at least once, followed by a colon, one space, and its frequency. List the letters in alphabetical order using lowercase letters. Finally, display the total number of English letters counted.

If the input contains no English letters, display only Total letters: 0. Do not use a map or collection class. Read the entire line from standard input without displaying an input prompt.

Question 4: Student Name Search

Write a Java program that searches a list of student names. The program first reads an integer n, representing the number of students. It then reads n names, one per line, and stores them in a String array. The final input line contains a search keyword.

Each name and the keyword contain between 1 and 30 English letters without spaces. Duplicate names are allowed.

A name matches if it contains the keyword anywhere, regardless of letter case. For example, the keyword an matches both Andy and Joanna.

Search the array and display every matching name in its original spelling and input order. Duplicate matching names must each be displayed. After listing the matches, display their total number using the label Matches: . If no names match, display Not found before the total.

Assume that $1 \leq n \leq 50$. Read from standard input without displaying input prompts.

Question 5: Word Array Reversal

Write a Java program that processes an array of words. The program first reads an integer n, representing the number of words. It then reads n words, one per line, and stores them in a String array.

Display the words in reverse input order. Before displaying each word, reverse its characters using a StringBuilder. Preserve the original uppercase and lowercase characters.

While processing the array, also count how many original words are palindromes. A word is a palindrome if it reads the same forward and backward when letter case is ignored. For example, Level is a palindrome, while Java is not. Every single-letter word is a palindrome.

Display each reversed word on a separate line. Finally, display the palindrome count using the label shown below.

Assume that $1 \leq n \leq 30$ and that each word contains between 1 and 30 English letters without spaces. Read from standard input without displaying input prompts.